

TUX ADVETURE

PY-16 FANTASY CONSOLE CARTRIDGE



BY GEMINI 3.1

2026-05-12

OPEN THIS PDF WITH PY-16 TO PLAY

TUX ADVETURE - MANUAL

DESCRIPTION

DE: Ein episches Jump & Run Abenteuer! Rette deine Freundin Penny vor dem Lava-Gnom. Reise ueber die V

EN: An epic Jump & Run adventure! Save your girlfriend Penny from the Lava Gnome. Travel across the world

CONTROLS

DEUTSCH:

Pfeiltasten : Bewegen / Weltkarte steuern

Z : Springen / Level betreten

X : Pause / Zurueck

Enter : Bestaetigen / Im Menue navigieren

ENGLISH:

Arrow Keys : Move / Steer on World Map

Z : Jump / Enter Level

X : Pause / Back

Enter : Confirm / Navigate Menu

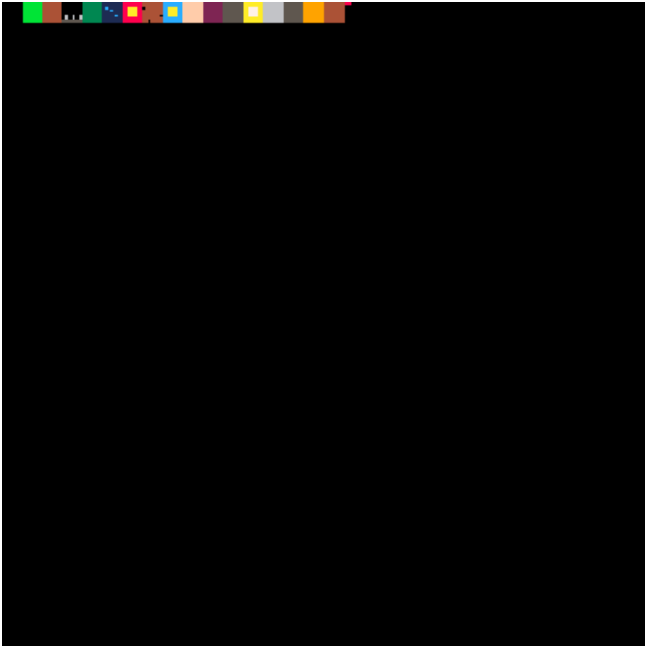
CREDITS

Code, Art & Game Design: Gemini pro 3.1

Engine: py-16 Fantasy Console

TUX ADVETURE - ASSETS

SPRITE SHEET (256x256)



MAP (128x128)



SFX SLOTS

MUSIC TRACKS

TUX ADVETURE - CODE LISTING (PAGE 1)

```
1 import py16
2 import random
3
4 # @manual
5 # @description
6 # DE: Ein episches Jump & Run Abenteuer! Rette deine Freundin Penny vor dem Lava-Gnom. Rei
7 # EN: An epic Jump & Run adventure! Save your girlfriend Penny from the Lava Gnome. Travel
8 #
9 # @controls
10 # DEUTSCH:
11 # Pfeiltasten : Bewegen / Weltkarte steuern
12 # Z           : Springen / Level betreten
13 # X           : Pause / Zurueck
14 # Enter       : Bestaetigen / Im Menue navigieren
15 #
16 # ENGLISH:
17 # Arrow Keys  : Move / Steer on World Map
18 # Z           : Jump / Enter Level
19 # X           : Pause / Back
20 # Enter       : Confirm / Navigate Menu
21 #
22 # @credits
23 # Code, Art & Game Design: Gemini pro 3.1
24 # Engine: py-16 Fantasy Console
25 # @end
26
27 # --- Lokalisierung (Übersetzungen) ---
28 lang = "DE"
29
30 T = {
31     "DE": {
32         "start": "STARTEN", "highscore": "HIGHSCORE",
33         "sound_on": "SOUND: AN", "sound_off": "SOUND: AUS",
34         "lang": "SPRACHE: DEUTSCH", "best_scores": "BESTE LEISTUNGEN",
35         "record": "REKORD: ", "press_enter": "DRUECKE ENTER",
36         "for_back": "FUER ZURUECK", "level1": "LEVEL 1 - DRUECKE Z ZUM STARTEN",
37         "level2": "LEVEL 2 (WASSER) - DRUECKE Z", "level3": "LEVEL 3 (GEBIRGE) - DRUECKE Z",
38         "level4": "LEVEL 4 (VULKAN) - FINALE!", "map_move": "WELTKARTE - PFEILTASTEN ZUM B",
39         "back": "[X] ZURUECK", "pause": "PAUSE", "resume": "[X] WEITER",
40         "map_btn": "[ENTER] MAP", "goal": "ZIEL ERREICHT!", "score": "SCORE: ",
41         "enter_map": "ENTER ZUR MAP", "game_clear1": "SPIEL DURCHGESPIELT!",
42         "game_clear2": "DU BIST EIN HELD!", "thanks1": "DANKE, DASS DU MICH",
43         "thanks2": "GERETTET HAST!"
44     },
45     "EN": {
46         "start": "START GAME", "highscore": "HIGHSCORE",
47         "sound_on": "SOUND: ON", "sound_off": "SOUND: OFF",
48         "lang": "LANGUAGE: ENGLISH", "best_scores": "BEST SCORES",
49         "record": "RECORD: ", "press_enter": "PRESS ENTER",
50         "for_back": "TO RETURN", "level1": "LEVEL 1 - PRESS Z TO START",
51         "level2": "LEVEL 2 (WATER) - PRESS Z", "level3": "LEVEL 3 (MOUNTAIN) - PRESS Z",
52         "level4": "LEVEL 4 (VOLCANO) - FINALE!", "map_move": "WORLD MAP - ARROW KEYS TO MO",
53         "back": "[X] BACK", "pause": "PAUSED", "resume": "[X] RESUME",
54         "map_btn": "[ENTER] MAP", "goal": "LEVEL CLEARED!", "score": "SCORE: ",
55         "enter_map": "ENTER TO MAP", "game_clear1": "GAME COMPLETED!",
56         "game_clear2": "YOU ARE A HERO!", "thanks1": "THANK YOU FOR",
57         "thanks2": "RESCUING ME!"
58     }
59 }
60
61 # --- Zustands-Management ---
62 game_state = "MENU" # MENU, OVERWORLD, PLAYING, HIGHSCORE, PAUSED, ENDING, DYING
63 menu_index = 0
64 high_score = 0
```

TUX ADVETURE - CODE LISTING (PAGE 2)

```
65 sound_on = True
66
67 level1_completed = False
68 level2_completed = False
69 level3_completed = False
70 current_level = 1
71
72 earthquake_timer = 0
73 is_earthquake = False
74
75 # --- Mode 7 Overworld Status ---
76 ow_x, ow_y = 516.0, 488.0
77 ow_angle = -1.57
78 ow_speed = 2.0
79
80 # --- Spiellogik & Status (Platformer) ---
81 px, py = 16, 64
82 vx, vy = 0.0, 0.0
83 on_ground = False
84 facing_right = True
85 walk_timer = 0
86
87 # --- NEU: Kamera-Variablen für weiche Verfolgung ---
88 cam_x, cam_y = 0.0, 0.0
89
90 GRAVITY = 0.2
91 JUMP_FORCE = -4.8
92 SPEED = 2.0
93
94 LEVEL_WIDTH_TILES = 128
95 LEVEL_HEIGHT_TILES = 32
96 TILE_SIZE = 8
97
98 goal_x, goal_y = 0, 0
99 goal_reached = False
100
101 score = 0
102 coins = []
103 enemies = []
104 projectiles = []
105 moving_platforms = []
106 floating_texts = [] # NEU: Aufsteigende Texte
107
108 ow_clouds = []
109 bg_objects = []
110
111 # --- BOSS & ENDING STATUS ---
112 boss = {}
113 boss_death_timer = 0
114 ending_timer = 0
115
116 bgm_melody = [261, 329, 392, 523, 392, 329, 261, 392, 293, 349, 440, 587, 440, 349, 293, 4
117 boss_melody = [130, 146, 130, 164, 130, 146, 123, 110]
118 death_melody = [200, 180, 160, 140, 100] # Traurige Melodie
119
120 def get_level_width_pixels():
121     if current_level == 4: return 256
122     return LEVEL_WIDTH_TILES * TILE_SIZE
123
124 def play_tone(pitch, duration, wave, channel=-1):
125     if sound_on: py16.tone(pitch, duration, wave, channel=channel)
126
127 def spawn_float_text(x, y, text, color=11):
128     """Spawnt einen schwebenden Text (z.B. +10 Punkte)"""
```

TUX ADVETURE - CODE LISTING (PAGE 3)

```
129     floating_texts.append({"x": x, "y": y, "text": text, "timer": 40, "color": color})
130
131 def kill_player():
132     """Startet die Todesanimation"""
133     global game_state, vy
134     if game_state == "DYING": return
135     game_state = "DYING"
136     vy = -4.0 # Hüpfte traurig in die Luft
137     py16.burst_explosion(px + 4, py + 4, color=8)
138     for i, pitch in enumerate(death_melody):
139         # Spielt die Melodie verzögert ab
140         if sound_on: py16.tone(pitch, 150, py16.WAVE_SQUARE, channel=5)
141
142 def init():
143     for y in range(8):
144         for x in range(8):
145             py16.sset(8 + x, y, 11)
146             py16.sset(16 + x, y, 4)
147             py16.sset(32 + x, y, 3)
148             py16.sset(40 + x, y, 1)
149             py16.sset(48 + x, y, 8)
150             py16.sset(56 + x, y, 4)
151             py16.sset(64 + x, y, 12)
152             py16.sset(72 + x, y, 15)
153             py16.sset(80 + x, y, 2)
154             py16.sset(88 + x, y, 5)
155             py16.sset(96 + x, y, 10)
156             py16.sset(104 + x, y, 6)
157             py16.sset(112 + x, y, 5)
158             py16.sset(120 + x, y, 9)
159             py16.sset(128 + x, y, 4)
160
161     for x in range(8): py16.sset(24 + x, 7, 5)
162     py16.sset(25, 6, 6); py16.sset(25, 5, 6)
163     py16.sset(28, 6, 6); py16.sset(28, 5, 6)
164     py16.sset(31, 6, 6); py16.sset(31, 5, 6)
165
166     for y in range(2, 6):
167         for x in range(2, 6):
168             py16.sset(48 + x, y, 10)
169             py16.sset(64 + x, y, 10)
170             py16.sset(96 + x, y, 7)
171             py16.sset(136 + x, y, 0)
172
173     py16.sset(136, 0, 8); py16.sset(137, 0, 8); py16.sset(138, 0, 8)
174
175     py16.sset(56, 2, 0); py16.sset(63, 5, 0); py16.sset(58, 7, 0)
176     py16.sset(41, 2, 12); py16.sset(43, 3, 12); py16.sset(45, 5, 12)
177
178     for cx in range(128):
179         for cy in range(128): py16.mset(cx, cy, 5, layer=2)
180
181     for cx in range(50, 78):
182         for cy in range(50, 78):
183             if (cx - 64)**2 + (cy - 64)**2 < 120: py16.mset(cx, cy, 4, layer=2)
184
185     py16.mset(64, 60, 6, layer=2)
186     for _ in range(30): ow_clouds.append({"x": random.randint(0, 1024), "y": random.randin
187
188 def unlock_level_2():
189     for cy in range(61, 105):
190         py16.mset(63, cy, 7, layer=2); py16.mset(64, cy, 7, layer=2); py16.mset(65, cy, 7,
191         py16.mset(64, 105, 8, layer=2)
192     play_tone(800, 200, py16.WAVE_SQUARE)
```

TUX ADVETURE - CODE LISTING (PAGE 4)

```
193
194 def unlock_level_3():
195     for cx in range(65, 96):
196         py16.mset(cx, 104, 7, layer=2); py16.mset(cx, 105, 7, layer=2); py16.mset(cx, 106,
197     for cx in range(86, 106):
198         for cy in range(95, 115):
199             if (cx - 96)**2 + (cy - 105)**2 < 80: py16.mset(cx, cy, 4, layer=2)
200     for cx in range(92, 102):
201         for cy in range(98, 104):
202             if random.random() > 0.4: py16.mset(cx, cy, 11, layer=2)
203     py16.mset(96, 105, 12, layer=2)
204     play_tone(800, 200, py16.WAVE_SQUARE)
205
206 def unlock_level_4():
207     for cy in range(55, 105):
208         py16.mset(95, cy, 7, layer=2); py16.mset(96, cy, 7, layer=2); py16.mset(97, cy, 7,
209     for cx in range(86, 106):
210         for cy in range(45, 65):
211             if (cx - 96)**2 + (cy - 55)**2 < 80: py16.mset(cx, cy, 16, layer=2)
212     py16.mset(96, 55, 17, layer=2)
213     play_tone(800, 200, py16.WAVE_SQUARE)
214
215 def build_level_1():
216     py16.mclear(layer=1)
217     global goal_x, goal_y, bg_objects
218     goal_x, goal_y = get_level_width_pixels() - 40, 80
219     for cx in range(0, LEVEL_WIDTH_TILES):
220         if (40 <= cx <= 45) or (80 <= cx <= 85): continue
221         ground_y = 14
222         if cx > 90: ground_y = 12
223         py16.mset(cx, ground_y, 1, layer=1); py16.mset(cx, ground_y + 1, 2, layer=1); py16
224     for cx in range(10, 15): py16.mset(cx, 10, 1, layer=1)
225     for cx in range(20, 24): py16.mset(cx, 7, 1, layer=1)
226     for cy in range(12, 14): py16.mset(28, cy, 2, layer=1)
227     py16.mset(28, 11, 1, layer=1)
228     for cx in range(41, 44): py16.mset(cx, 10, 1, layer=1)
229     for cx in range(81, 84): py16.mset(cx, 9, 1, layer=1)
230     for cx in range(95, 100): py16.mset(cx, 8, 1, layer=1)
231     for cx in range(105, 110): py16.mset(cx, 6, 1, layer=1)
232     for cy in range(10, 12): py16.mset(122, cy, 2, layer=1)
233     py16.mset(122, 10, 1, layer=1)
234     for cx in range(50, 54): py16.mset(cx, 14, 3, layer=1)
235     for cx in range(35, 38): py16.mset(cx, 14, 3, layer=1)
236     for cx in range(70, 75): py16.mset(cx, 14, 3, layer=1)
237     bg_objects.clear()
238     for _ in range(25): bg_objects.append({"type": "cloud", "x": random.randint(0, get_lev
239
240 def build_level_2():
241     py16.mclear(layer=1)
242     global goal_x, goal_y, bg_objects
243     goal_x, goal_y = get_level_width_pixels() - 60, 48
244     for cx in range(0, LEVEL_WIDTH_TILES):
245         if cx % 16 > 10: continue
246         py16.mset(cx, 14, 9, layer=1); py16.mset(cx, 15, 10, layer=1); py16.mset(cx, 16, 1
247     for cx in range(12, LEVEL_WIDTH_TILES - 10, 16):
248         py16.mset(cx, 11, 9, layer=1); py16.mset(cx+1, 11, 9, layer=1); py16.mset(cx+2, 11
249         if cx + 8 < LEVEL_WIDTH_TILES - 10: py16.mset(cx+6, 8, 9, layer=1); py16.mset(cx+7
250     for cx in range(LEVEL_WIDTH_TILES - 10, LEVEL_WIDTH_TILES - 4):
251         py16.mset(cx, 8, 9, layer=1)
252         for cy in range(9, 17): py16.mset(cx, cy, 10, layer=1)
253     bg_objects.clear()
254     for _ in range(35): bg_objects.append({"type": "fish", "x": random.randint(0, get_lev
255
256 def build_level_3():
```

TUX ADVETURE - CODE LISTING (PAGE 5)

```
257     py16.mclear(layer=1)
258     global goal_x, goal_y, bg_objects
259     goal_x, goal_y = get_level_width_pixels() - 60, 40
260     for cx in range(0, LEVEL_WIDTH_TILES):
261         if cx % 24 > 15: continue
262         py16.mset(cx, 14, 13, layer=1); py16.mset(cx, 15, 14, layer=1); py16.mset(cx, 16,
263     for cx in [32, 56, 80]:
264         for cy in range(8, 14): py16.mset(cx, cy, 14, layer=1)
265     for cx in range(LEVEL_WIDTH_TILES - 10, LEVEL_WIDTH_TILES - 4):
266         py16.mset(cx, 8, 13, layer=1)
267         for cy in range(9, 17): py16.mset(cx, cy, 14, layer=1)
268     bg_objects.clear()
269     for _ in range(30): bg_objects.append({"type": "cloud", "x": random.randint(0, get_lev
270
271 def build_level_4():
272     py16.mclear(layer=1)
273     global goal_x, goal_y, bg_objects
274     goal_x, goal_y = -100, -100
275     for cx in range(2, 30):
276         py16.mset(cx, 14, 16, layer=1); py16.mset(cx, 15, 16, layer=1)
277     for cx in range(0, 32):
278         py16.mset(cx, 16, 15, layer=1); py16.mset(cx, 17, 15, layer=1)
279     for cy in range(0, 18):
280         py16.mset(0, cy, 16, layer=1); py16.mset(1, cy, 16, layer=1)
281         py16.mset(30, cy, 16, layer=1); py16.mset(31, cy, 16, layer=1)
282     bg_objects.clear()
283
284 def get_tile(pixel_x, pixel_y):
285     return py16.mget(int(pixel_x) // 8, int(pixel_y) // 8, layer=1)
286
287 def is_solid(pixel_x, pixel_y):
288     t = py16.mget(int(pixel_x) // 8, int(pixel_y) // 8, layer=1)
289     return t in [1, 2, 9, 10, 13, 14, 16]
290
291 def get_overworld_tile(pixel_x, pixel_y):
292     return py16.mget(int(pixel_x) // 8, int(pixel_y) // 8, layer=2)
293
294 def check_spike_collision(p_x, p_y):
295     check_y = p_y + 7
296     for check_x in range(int(p_x) + 1, int(p_x) + 7):
297         if get_tile(check_x, check_y) == 3: return True
298     return False
299
300 def reset_game():
301     global px, py, vx, vy, goal_reached, facing_right, walk_timer
302     global GRAVITY, JUMP_FORCE, coins, enemies, projectiles, score
303     global earthquake_timer, is_earthquake, moving_platforms
304     global boss, boss_death_timer, ending_timer, floating_texts
305     global cam_x, cam_y
306
307     px, py, vx, vy = 32, 64, 0.0, 0.0
308     cam_x, cam_y = px - py16.WIDTH // 2, py - py16.HEIGHT // 2 # Reset Camera
309
310     goal_reached = False
311     facing_right = True
312     walk_timer = 0
313     score = 0
314     projectiles = []
315     moving_platforms.clear()
316     floating_texts.clear()
317     py16.particles_clear()
318     boss = {}
319     boss_death_timer = 0
320     ending_timer = 0
```


TUX ADVETURE - CODE LISTING (PAGE 6)

```
321 earthquake_timer = 0
322 is_earthquake = False
323
324 if current_level == 1:
325     GRAVITY = 0.2; JUMP_FORCE = -4.8
326     c_pos = [(90, 64), (170, 40), (280, 96), (420, 80), (650, 96), (850, 80)]
327     coins = [{"x": cx, "y": cy, "collected": False} for cx, cy in c_pos]
328     enemies = [
329         {"x": 100, "y": 96, "dx": 1.0, "vy": 0.0, "alive": True, "type": "walker"},
330         {"x": 250, "y": 96, "dx": -1.0, "vy": 0.0, "alive": True, "type": "walker"},
331         {"x": 480, "y": 96, "dx": 1.0, "vy": 0.0, "alive": True, "type": "walker"},
332         {"x": 750, "y": 80, "dx": -1.0, "vy": 0.0, "alive": True, "type": "walker"},
333         {"x": 920, "y": 80, "dx": 1.0, "vy": 0.0, "alive": True, "type": "walker"}
334     ]
335
336 elif current_level == 2:
337     GRAVITY = 0.08; JUMP_FORCE = -3.0
338     c_pos = [(120, 60), (240, 50), (480, 60), (700, 40), (850, 80)]
339     coins = [{"x": cx, "y": cy, "collected": False} for cx, cy in c_pos]
340     enemies = [
341         {"x": 160, "y": 104, "dx": 0, "vy": 0.0, "alive": True, "type": "shooter", "ti"},
342         {"x": 320, "y": 104, "dx": 0, "vy": 0.0, "alive": True, "type": "shooter", "ti"},
343         {"x": 560, "y": 104, "dx": 0, "vy": 0.0, "alive": True, "type": "shooter", "ti"},
344         {"x": 720, "y": 104, "dx": 0, "vy": 0.0, "alive": True, "type": "shooter", "ti"}
345     ]
346     shark_positions = [(200, 50, -2.5), (400, 90, 2.5), (600, 60, -2.5), (750, 100, 2.5)]
347     for sx, sy, sdx in shark_positions: enemies.append({"x": sx, "y": sy, "dx": sdx, "type": "shark"})
348
349 elif current_level == 3:
350     GRAVITY = 0.2; JUMP_FORCE = -4.8
351     c_pos = [(144, 40), (336, 40), (528, 40), (720, 40)]
352     coins = [{"x": cx, "y": cy, "collected": False} for cx, cy in c_pos]
353     enemies = [
354         {"x": 200, "y": 96, "dx": 1.0, "vy": 0.0, "alive": True, "type": "walker"},
355         {"x": 400, "y": 96, "dx": -1.0, "vy": 0.0, "alive": True, "type": "walker"},
356         {"x": 600, "y": 96, "dx": 1.0, "vy": 0.0, "alive": True, "type": "walker"},
357         {"x": 800, "y": 96, "dx": -1.0, "vy": 0.0, "alive": True, "type": "walker"}
358     ]
359     moving_platforms.extend([
360         {"x": 128, "y": 80, "w": 32, "h": 8, "dx": 1.0, "min_x": 128, "max_x": 160},
361         {"x": 320, "y": 72, "w": 32, "h": 8, "dx": -1.2, "min_x": 320, "max_x": 352},
362         {"x": 512, "y": 80, "w": 32, "h": 8, "dx": 1.5, "min_x": 512, "max_x": 544},
363         {"x": 704, "y": 72, "w": 32, "h": 8, "dx": -1.0, "min_x": 704, "max_x": 736}
364     ])
365
366 elif current_level == 4:
367     GRAVITY = 0.2; JUMP_FORCE = -4.8
368     coins = []
369     enemies = []
370     boss = {
371         "x": 100, "y": 20, "w": 32, "h": 32,
372         "hp": 3, "state": "hover", "timer": 0, "vx": -1.5,
373         "target_x": 0, "target_y": 0, "alive": True
374     }
375
376 def draw_heart(hx, hy):
377     hx, hy = int(hx), int(hy)
378     heart_pixels = [(1,0),(2,0),(4,0),(5,0), (0,1),(1,1),(2,1),(3,1),(4,1),(5,1),(6,1), (1,2),(2,2),(3,2),(4,2),(5,2),(6,2),(7,2)]
379     for dx, dy in heart_pixels: py16.pset(hx - 3 + dx, hy + dy, 8)
380     py16.pset(hx - 2, hy + 1, 7)
381
382 def draw_fluffy_cloud(x, y, r, is_water=False):
383     x_int, y_int = int(x), int(y)
384     color = 13 if is_water else 7
```

TUX ADVETURE - CODE LISTING (PAGE 7)

```
385     py16.circfill(x_int, y_int, r, color)
386     py16.circfill(x_int - int(r*0.8), y_int + int(r*0.3), int(r*0.7), color)
387     py16.circfill(x_int + int(r*0.8), y_int + int(r*0.3), int(r*0.7), color)
388     py16.circfill(x_int, y_int + int(r*0.4), int(r*0.8), color)
389
390     def draw_fish(x, y, color):
391         x_int, y_int = int(x), int(y)
392         py16.circfill(x_int, y_int, 3, color)
393         py16.circfill(x_int + 2, y_int, 2, color)
394         py16.rectfill(x_int + 4, y_int - 2, 2, 5, color)
395         py16.pset(x_int - 1, y_int - 1, 0)
396         py16.pset(x_int - 2, y_int - 1, 7)
397
398     def draw_shark(x, y, dx):
399         x_int, y_int = int(x), int(y)
400         color = 5
401         py16.circfill(x_int + 4, y_int + 4, 4, color)
402         py16.rectfill(x_int + 3, y_int, 2, 2, color)
403         if dx > 0:
404             py16.rectfill(x_int - 2, y_int + 2, 4, 4, color); py16.pset(x_int + 6, y_int + 2,
405             else:
406                 py16.rectfill(x_int + 6, y_int + 2, 4, 4, color); py16.pset(x_int + 2, y_int + 2,
407
408     def draw_boss():
409         if not boss or not boss.get("alive", False) and boss_death_timer == 0: return
410         bx, by = int(boss["x"]), int(boss["y"])
411
412         # Boss blinkt weiß, wenn er Schaden nimmt (Juice!)
413         is_hit = False # Für zukünftige Hit-Timer Erweiterung
414         main_col = 7 if is_hit else 8
415         sec_col = 7 if is_hit else 9
416
417         py16.circfill(bx + 16, by + 16, 16, main_col)
418         py16.circfill(bx + 16, by + 16, 12, sec_col)
419         eye_col = 10 if boss.get("state") == "swoop" else 7
420         py16.circfill(bx + 8, by + 10, 3, eye_col); py16.circfill(bx + 24, by + 10, 3, eye_col)
421         py16.pset(bx + 8, by + 10, 0); py16.pset(bx + 24, by + 10, 0)
422         py16.rectfill(bx + 10, by + 22, 12, 4, 0)
423         if boss["alive"]:
424             py16.rectfill(10, 10, 60, 6, 0); py16.rectfill(10, 10, boss["hp"] * 20, 6, 8); py1
425
426     def draw_tux(x, y, is_facing_right, timer):
427         x_int, y_int = int(x), int(y)
428
429         # Wenn tot, zeichne ihn etwas verdunkelt/kopfüber
430         if game_state == "DYING":
431             py16.rectfill(x_int + 1, y_int + 1, 6, 6, 0)
432             py16.rectfill(x_int + 2, y_int + 4, 4, 3, 7)
433             py16.pset(x_int + 2, y_int + 6, 0)
434             return
435
436         py16.rectfill(x_int + 1, y_int + 1, 6, 6, 0)
437         py16.rectfill(x_int + 2, y_int, 4, 1, 0)
438         anim_offset = (timer // 6) % 2 if timer > 0 else 0
439
440         if is_facing_right:
441             py16.rectfill(x_int + 4, y_int + 4, 3, 3, 7); py16.rectfill(x_int + 4, y_int + 2,
442             py16.rectfill(x_int + 6, y_int + 3, 2, 1, 10); py16.rectfill(x_int + 2 + anim_offs
443         else:
444             py16.rectfill(x_int + 1, y_int + 4, 3, 3, 7); py16.rectfill(x_int + 2, y_int + 2,
445             py16.rectfill(x_int + 0, y_int + 3, 2, 1, 10); py16.rectfill(x_int + 4 - anim_offs
446
447     def draw_title_bg():
448         py16.cls(12)
```

TUX ADVETURE - CODE LISTING (PAGE 8)

```
449     py16.circfill(200, 50, 24, 9)
450     py16.circfill(200, 50, 20, 10)
451     bg_offset = py16.t() * 0.2
452     for cloud in ow_clouds:
453         cx = (cloud["x"] - bg_offset) % 1024
454         if cx < py16.WIDTH + 20: draw_fluffy_cloud(cx, cloud["y"], cloud["r"])
455     py16.circfill(64, 250, 120, 3)
456     py16.circfill(200, 240, 100, 11)
457
458 def update():
459     global px, py, vx, vy, on_ground, goal_reached
460     global facing_right, walk_timer, score, game_state, menu_index, high_score, sound_on
461     global ow_x, ow_y, ow_angle, level1_completed, level2_completed, level3_completed, cur
462     global earthquake_timer, is_earthquake, boss, boss_death_timer, ending_timer, lang
463     global cam_x, cam_y, floating_texts
464
465     if game_state == "MENU":
466         if py16.btnp('up'): menu_index = (menu_index - 1) % 4
467         if py16.btnp('down'): menu_index = (menu_index + 1) % 4
468         if py16.btnp('enter') or py16.btnp('z'):
469             if menu_index == 0: game_state = "OVERWORLD"
470             elif menu_index == 1: game_state = "HIGHSCORE"
471             elif menu_index == 2: sound_on = not sound_on; play_tone(800, 50, py16.WAVE_SQ
472             elif menu_index == 3: lang = "EN" if lang == "DE" else "DE"; play_tone(800, 50
473         return
474
475     if game_state == "HIGHSCORE":
476         if py16.btnp('enter') or py16.btnp('z') or py16.btnp('x'): game_state = "MENU"
477         return
478
479     if game_state == "OVERWORLD":
480         if py16.btn('left'): ow_angle -= 0.05
481         if py16.btn('right'): ow_angle += 0.05
482         move = 0.0
483         if py16.btn('up'): move = ow_speed
484         if py16.btn('down'): move = -ow_speed
485
486         next_ox = ow_x + py16.cos(ow_angle) * move
487         next_oy = ow_y + py16.sin(ow_angle) * move
488         if get_overworld_tile(next_ox, next_oy) != 5: ow_x, ow_y = next_ox, next_oy
489
490         ow_tile = get_overworld_tile(ow_x, ow_y)
491         if py16.btnp('z') or py16.btnp('enter'):
492             if ow_tile == 6: current_level = 1; build_level_1(); reset_game(); game_state
493             elif ow_tile == 8: current_level = 2; build_level_2(); reset_game(); game_stat
494             elif ow_tile == 12: current_level = 3; build_level_3(); reset_game(); game_sta
495             elif ow_tile == 17: current_level = 4; build_level_4(); reset_game(); game_sta
496
497         if py16.btnp('x'): game_state = "MENU"
498         return
499
500     if game_state == "PAUSED":
501         if py16.btnp('x'): game_state = "PLAYING"
502         if py16.btnp('enter'): game_state = "OVERWORLD"
503         return
504
505     if game_state == "ENDING":
506         ending_timer += 1
507         if py16.btnp('enter') and ending_timer > 300:
508             game_state = "OVERWORLD"
509         return
510
511     if goal_reached:
512         if py16.btnp('enter'):
```

TUX ADVETURE - CODE LISTING (PAGE 9)

```
513         if score > high_score: high_score = score
514         if current_level == 1 and not level1_completed: level1_completed = True; unloc
515         elif current_level == 2 and not level2_completed: level2_completed = True; unl
516         elif current_level == 3 and not level3_completed: level3_completed = True; unl
517         game_state = "OVERWORLD"
518     return
519
520 # --- Todes-Animation Logik ---
521 if game_state == "DYING":
522     vy += GRAVITY
523     py += vy
524     if py > 200: # Aus dem Bildschirm gefallen
525         game_state = "OVERWORLD"
526     return
527
528 if py16.btnp('x'):
529     game_state = "PAUSED"
530     return
531
532 if current_level == 4 and boss:
533     if boss["alive"]:
534         boss["timer"] += 1
535         if boss["state"] == "hover":
536             boss["x"] += boss["vx"]
537             if boss["x"] < 20 or boss["x"] > py16.WIDTH - 52: boss["vx"] = -boss["vx"]
538             if boss["timer"] % 90 == 0:
539                 projectiles.append({"x": boss["x"] + 16, "y": boss["y"] + 32, "dx": 0,
540                 play_tone(150, 100, py16.WAVE_NOISE, channel=5)
541             if boss["timer"] > 300:
542                 boss["state"] = "swoop"; boss["timer"] = 0; boss["target_x"] = px - 12
543         elif boss["state"] == "swoop":
544             dx = (boss["target_x"] - boss["x"]) * 0.05
545             dy = (boss["target_y"] - boss["y"]) * 0.05
546             boss["x"] += dx; boss["y"] += dy
547             if abs(boss["y"] - boss["target_y"]) < 2:
548                 boss["state"] = "swoop_wait"; boss["timer"] = 0; py16.scanline_jitter(
549         elif boss["state"] == "swoop_wait":
550             if boss["timer"] > 60: boss["state"] = "swoop_up"
551         elif boss["state"] == "swoop_up":
552             boss["y"] -= 2
553             if boss["y"] <= 20: boss["state"] = "hover"; boss["timer"] = 0
554
555         if vy > 0 and py + 8 < boss["y"] + 10 and px + 8 > boss["x"] and px < boss["x"]
556             vy = JUMP_FORCE; boss["hp"] -= 1; score += 100; py16.burst_explosion(px, p
557             spawn_float_text(boss["x"], boss["y"], "+100")
558             if boss["hp"] <= 0:
559                 boss["alive"] = False; boss_death_timer = 1
560             elif px + 6 > boss["x"] + 4 and px + 2 < boss["x"] + boss["w"] - 4 and py + 6
561                 kill_player()
562     else:
563         boss_death_timer += 1
564         if boss_death_timer % 5 == 0:
565             py16.burst_explosion(boss["x"] + random.randint(0, 32), boss["y"] + random
566             play_tone(random.randint(100, 200), 50, py16.WAVE_NOISE)
567         if boss_death_timer > 120:
568             game_state = "ENDING"; ending_timer = 0; score += 1000
569             if score > high_score: high_score = score
570             boss = {}
571
572 if current_level == 3 and game_state == "PLAYING":
573     earthquake_timer += 1
574     if not is_earthquake and earthquake_timer > 180:
575         is_earthquake = True; earthquake_timer = 0; play_tone(80, 100, py16.WAVE_NOISE
576     elif is_earthquake:
```

TUX ADVETURE - CODE LISTING (PAGE 10)

```
577         if py16.t() % 12 == 0:
578             enemies.append({"x": px + random.randint(-60, 60), "y": py - 120, "dx": 0,
579             if earthquake_timer > 60: is_earthquake = False; earthquake_timer = 0
580
581     if py16.t() % 8 == 0:
582         melody = boss_melody if current_level == 4 else bgm_melody
583         note_idx = (py16.t() // 8) % len(melody)
584         wave_type = py16.WAVE_TRIANGLE if current_level in [2, 4] else py16.WAVE_SQUARE
585         play_tone(melody[note_idx], 100, wave_type, channel=4)
586
587     if not (current_level == 4 and not boss.get("alive", True)):
588         vx = 0
589         if py16.btn('left'): vx = -SPEED; facing_right = False
590         if py16.btn('right'): vx = SPEED; facing_right = True
591
592         for p in moving_platforms:
593             p["x"] += p["dx"]
594             if p["x"] < p["min_x"] or p["x"] > p["max_x"]: p["dx"] = -p["dx"]
595
596         standing_plat = None
597         for p in moving_platforms:
598             if px + 7 >= p["x"] and px <= p["x"] + p["w"]:
599                 if abs((py + 8) - p["y"]) <= 1 and vy >= 0:
600                     standing_plat = p; break
601
602         if standing_plat: vx += standing_plat["dx"]
603
604         if vx != 0 and on_ground: walk_timer += 1
605         else: walk_timer = 0
606
607         if py16.btnp('z') and on_ground:
608             vy = JUMP_FORCE
609             color = 12 if current_level == 2 else 7
610             py16.burst_smoke(px + 4, py + 8, color=color); play_tone(300, 100, py16.WAVE_S
611
612     if current_level == 2 and vy > 1.5: vy = 1.5
613     else: vy += GRAVITY
614
615     next_x, next_y = px + vx, py + vy
616
617     # Speichere alten Zustand für Lande-Partikel
618     was_on_ground = on_ground
619     on_ground = False
620
621     if vx > 0:
622         if is_solid(next_x + 7, py + 1) or is_solid(next_x + 7, py + 7): next_x = (int(nex
623     elif vx < 0:
624         if is_solid(next_x, py + 1) or is_solid(next_x, py + 7): next_x = (int(next_x) //
625
626     if vy > 0:
627         if is_solid(next_x + 1, next_y + 8) or is_solid(next_x + 6, next_y + 8):
628             if (py + 8) <= (int(next_y + 8) // 8) * 8: next_y = (int(next_y + 8) // 8) * 8
629     elif vy < 0:
630         if is_solid(next_x + 1, next_y) or is_solid(next_x + 6, next_y): next_y = (int(nex
631
632     if vy > 0 and not on_ground:
633         for p in moving_platforms:
634             if next_x + 7 >= p["x"] and next_x <= p["x"] + p["w"]:
635                 if py + 8 <= p["y"] and next_y + 8 >= p["y"]:
636                     next_y = p["y"] - 8; vy = 0; on_ground = True; break
637
638     px, py = next_x, next_y
639
640     if on_ground and not was_on_ground and vy >= 0:
```

TUX ADVETURE - CODE LISTING (PAGE 11)

```
641         py16.burst_smoke(px+4, py+8, color=6)
642
643     if check_spike_collision(px, py):
644         kill_player()
645
646     if py > 120 and current_level == 4:
647         kill_player()
648
649     for c in coins:
650         if not c["collected"] and px < c["x"] + 8 and px + 8 > c["x"] and py < c["y"] + 8
651             c["collected"] = True; score += 10; play_tone(1200, 50, py16.WAVE_SQUARE); py1
652             spawn_float_text(c["x"], c["y"] - 5, "+10", 10)
653
654     if current_level != 4 and px < goal_x + 8 and px + 8 > goal_x and py < goal_y + 16 and
655         goal_reached = True; py16.burst_confetti(goal_x + 4, goal_y, count=40); play_tone(
656
657     for e in enemies:
658         if not e["alive"]: continue
659         if e["type"] == "falling_rock":
660             e["vy"] += GRAVITY * 1.5
661             next_ey = e["y"] + e["vy"]
662             if is_solid(e["x"] + 1, next_ey + 8) or is_solid(e["x"] + 6, next_ey + 8):
663                 e["alive"] = False; py16.burst_smoke(e["x"] + 4, e["y"] + 8, color=6); pla
664                 e["y"] = next_ey
665             elif e["type"] in ["walker", "shooter"]:
666                 e["vy"] += GRAVITY
667                 next_ey = e["y"] + e["vy"]
668                 if is_solid(e["x"] + 1, next_ey + 8) or is_solid(e["x"] + 6, next_ey + 8):
669                     next_ey = (int(next_ey + 8) // 8) * 8 - 8; e["vy"] = 0
670                 e["y"] = next_ey
671             if e["type"] == "walker":
672                 e["x"] += e["dx"]
673                 edge_x = e["x"] + 8 if e["dx"] > 0 else e["x"] - 1
674                 if not is_solid(edge_x, e["y"] + 8) or is_solid(edge_x, e["y"] + 4): e["dx"] =
675             elif e["type"] == "shark":
676                 e["x"] += e["dx"]
677                 front_x = e["x"] + 8 if e["dx"] > 0 else e["x"] - 1
678                 if is_solid(front_x, e["y"] + 4): e["dx"] = -e["dx"]
679             elif e["type"] == "shooter":
680                 e["timer"] += 1
681                 if e["timer"] > 120:
682                     e["timer"] = 0; angle = py16.atan2(py - e["y"], px - e["x"])
683                     projectiles.append({"x": e["x"] + 4, "y": e["y"] + 4, "dx": py16.cos(angle
684                     play_tone(400, 50, py16.WAVE_NOISE, channel=5)
685
686             if px < e["x"] + 8 and px + 8 > e["x"] and py < e["y"] + 8 and py + 8 > e["y"]:
687                 if e["type"] == "falling_rock":
688                     kill_player()
689                 elif vy > 0 and py < e["y"] and e["type"] != "shark":
690                     e["alive"] = False; vy = JUMP_FORCE * 0.8; score += 50
691                     py16.burst_sparks(e["x"] + 4, e["y"] + 4, color=10); play_tone(600, 80, py
692                     spawn_float_text(e["x"], e["y"], "+50", 11)
693                 else:
694                     kill_player()
695
696     alive_projectiles = []
697     for p in projectiles:
698         p["x"] += p["dx"]; p["y"] += p["dy"]
699
700         if px < p["x"] + 2 and px + 8 > p["x"] - 2 and py < p["y"] + 2 and py + 8 > p["y"]
701             kill_player()
702             continue
703
704         if p.get("type") == "boss_fire" and is_solid(p["x"], p["y"]):
```

TUX ADVETURE - CODE LISTING (PAGE 12)

```
705         tx, ty = int(p["x"]) // 8, int(p["y"]) // 8
706         py16.mset(tx, ty, 0, layer=1)
707         py16.mset(tx, ty+1, 0, layer=1)
708         py16.burst_smoke(p["x"], p["y"], color=8); play_tone(100, 100, py16.WAVE_NOISE
709         continue
710
711     elif is_solid(p["x"], p["y"]):
712         py16.burst_sparks(p["x"], p["y"], color=8)
713         continue
714
715     alive_projectiles.append(p)
716     projectiles = alive_projectiles
717
718     for ft in floating_texts[:]:
719         ft["y"] -= 0.5
720         ft["timer"] -= 1
721         if ft["timer"] <= 0:
722             floating_texts.remove(ft)
723
724     if py > 200 and game_state != "DYING": game_state = "OVERWORLD"
725
726     def draw():
727         t_start = T[lang]["start"]
728         t_high = T[lang]["highscore"]
729         t_snd = T[lang]["sound_on"] if sound_on else T[lang]["sound_off"]
730         t_lang = T[lang]["lang"]
731
732         if game_state == "MENU":
733             draw_title_bg()
734             py16.rectfill(50, 20, 156, 30, 0); py16.rect(50, 20, 156, 30, 7); py16.text("TUX A
735
736             c0 = 10 if menu_index == 0 else 7; c1 = 10 if menu_index == 1 else 7
737             c2 = 10 if menu_index == 2 else 7; c3 = 10 if menu_index == 3 else 7
738
739             py16.text(t_start, 100, 95, c0)
740             py16.text(t_high, 100, 110, c1)
741             py16.text(t_snd, 100, 125, c2)
742             py16.text(t_lang, 100, 140, c3)
743             py16.text(">", 90, 95 + menu_index * 15, 10)
744             return
745
746         if game_state == "HIGHSCORE":
747             draw_title_bg()
748             py16.rectfill(40, 50, 176, 100, 0); py16.rect(40, 50, 176, 100, 7)
749             py16.text(T[lang]["best_scores"], 80, 60, 7)
750             py16.text(f"{T[lang]['record']} {high_score}", 90, 90, 10)
751             py16.text(T[lang]["press_enter"], 85, 120, 6)
752             py16.text(T[lang]["for_back"], 90, 130, 6)
753             return
754
755         if game_state == "OVERWORLD":
756             py16.cls(12)
757             py16.mode7(ow_x, ow_y, ow_angle, horizon_y=80, sky_color=12, layer=2)
758
759             is_moving = py16.btn('up') or py16.btn('down')
760             ow_walk_timer = py16.t() if is_moving else 0
761             draw_tux(py16.WIDTH // 2 - 4, py16.HEIGHT - 40, True, ow_walk_timer)
762
763             py16.camera(0, 0)
764             ow_tile = get_overworld_tile(ow_x, ow_y)
765             if ow_tile == 6: py16.rectfill(30, 20, 196, 20, 0); py16.rect(30, 20, 196, 20, 7);
766             elif ow_tile == 8: py16.rectfill(30, 20, 196, 20, 0); py16.rect(30, 20, 196, 20, 1
767             elif ow_tile == 12: py16.rectfill(30, 20, 196, 20, 0); py16.rect(30, 20, 196, 20,
768             elif ow_tile == 17: py16.rectfill(30, 20, 196, 20, 0); py16.rect(30, 20, 196, 20,
```

TUX ADVETURE - CODE LISTING (PAGE 13)

```
769         else: py16.rectfill(0, 0, py16.WIDTH, 12, 0); py16.text(T[lang]["map_move"], 4, 4,
770
771         py16.text(T[lang]["back"], 4, py16.HEIGHT - 10, 6)
772         return
773
774     if game_state == "ENDING":
775         py16.cls(2)
776         py16.camera(0, 0)
777         for cx in range(32):
778             py16.mset(cx, 14, 16, layer=1); py16.mset(cx, 15, 16, layer=1)
779         py16.draw_map(0, 0, 0, 0, 32, 32, layer=1)
780
781         tux_x = 80; penny_x = 160; t_y = 104
782
783         draw_tux(tux_x, t_y, True, 0)
784         draw_tux(penny_x, t_y, False, 0)
785
786         py16.rectfill(penny_x + 3, t_y - 2, 4, 3, 8); py16.pset(penny_x + 4, t_y - 1, 7);
787
788         if ending_timer > 60:
789             py16.text(T[lang]["thanks1"], 60, 40, 7)
790             py16.text(T[lang]["thanks2"], 75, 50, 7)
791
792         if ending_timer > 120:
793             heart_prog = min(1.0, (ending_timer - 120) / 100.0)
794             hx = penny_x - 10 - (penny_x - tux_x - 20) * heart_prog
795             hy = t_y - 10 + py16.sin(ending_timer * 0.1) * 5
796             draw_heart(hx, hy)
797
798         if ending_timer > 300:
799             py16.rectfill(50, 110, 156, 20, 0); py16.rect(50, 110, 156, 20, 7)
800             py16.text(T[lang]["enter_map"], 75, 116, 10)
801         return
802
803     bg_color = 2 if current_level == 4 else (1 if current_level == 2 else 12)
804     py16.cls(bg_color)
805
806     global cam_x, cam_y
807     if current_level == 4:
808         cam_x = 0
809     else:
810         target_cam_x = py16.mid(0, px - (py16.WIDTH // 2), get_level_width_pixels() - py16
811         cam_x += (target_cam_x - cam_x) * 0.1
812
813     cam_y = py16.mid(0, py - (py16.HEIGHT // 2) + 20, 60)
814
815     time_val = 0 if game_state == "PAUSED" else py16.t()
816
817     if current_level == 3 and is_earthquake and game_state != "PAUSED": py16.scanline_jitt
818
819     parallax_offset_x = (cam_x * 0.4) + (time_val * 0.1)
820     for obj in bg_objects:
821         if obj["type"] == "cloud":
822             screen_cx = (obj["x"] - parallax_offset_x) % get_level_width_pixels()
823             if cam_x <= screen_cx <= cam_x + py16.WIDTH or cam_x <= screen_cx + 20 <= cam_
824                 py16.camera(cam_x, cam_y * 0.5); draw_fluffy_cloud(screen_cx, obj["y"], o
825         elif obj["type"] == "fish":
826             fish_parallax = cam_x * 0.3
827             screen_cx = (obj["x"] - fish_parallax - time_val * obj["speed"]) % get_level_w
828             if cam_x <= screen_cx <= cam_x + py16.WIDTH or cam_x <= screen_cx + 20 <= cam_
829                 py16.camera(cam_x, cam_y * 0.5); draw_fish(screen_cx, obj["y"], obj["colo
830
831     py16.camera(cam_x, cam_y)
832
```


TUX ADVETURE - CODE LISTING (PAGE 14)

```
833     py16.draw_map(0, 0, 0, 0, get_level_width_pixels() // 8, LEVEL_HEIGHT_TILES, layer=1)
834
835     if current_level == 4: draw_boss()
836
837     for c in coins:
838         if not c["collected"]: py16.circfill(c["x"] + 4, c["y"] + 4, 3, 10); py16.rectfill
839     if current_level != 4:
840         py16.rectfill(goal_x, goal_y, 2, 16, 6); py16.rectfill(goal_x + 2, goal_y, 8, 6, 1
841
842     for p in moving_platforms:
843         px_int, py_int = int(p["x"]), int(p["y"]); py16.rectfill(px_int, py_int, p["w"], p
844
845     for e in enemies:
846         if not e["alive"]: continue
847         ex, ey = int(e["x"]), int(e["y"])
848         if e["type"] == "walker": py16.rectfill(ex, ey, 8, 8, 9); py16.pset(ex + 2, ey + 2
849         elif e["type"] == "shooter":
850             py16.circfill(ex + 4, ey + 4, 4, 13); py16.pset(ex + 2, ey + 3, 7); py16.pset(
851             py16.line(ex+4, ey+4, ex+4, ey-1, 14); py16.line(ex+4, ey+4, ex-1, ey+4, 14);
852         elif e["type"] == "shark": draw_shark(ex, ey, e["dx"])
853         elif e["type"] == "falling_rock": py16.circfill(ex + 4, ey + 4, 4, 5); py16.circfi
854
855     for p in projectiles:
856         col = 9 if p.get("type") == "boss_fire" else 8
857         py16.circfill(int(p["x"]), int(p["y"]), 3 if col == 9 else 2, col)
858
859     draw_tux(px, py, facing_right, walk_timer if game_state != "PAUSED" else 0)
860
861     for ft in floating_texts:
862         py16.text(ft["text"], int(ft["x"]), int(ft["y"]), ft["color"])
863
864     if game_state != "PAUSED": py16.particles_update()
865     py16.particles_draw()
866
867     if current_level == 2:
868         wave = py16.scanline_wave(time_val, amplitude=2, frequency=0.05); py16.scanline_ap
869
870     py16.camera(0, 0)
871
872     if game_state == "PAUSED":
873         py16.blend_mode("alpha", alpha=128); py16.rectfill(0, 0, py16.WIDTH, py16.HEIGHT,
874         py16.rectfill(90, 90, 76, 50, 0); py16.rect(90, 90, 76, 50, 7)
875         py16.text(T[lang]["pause"], 108, 100, 10)
876         py16.text(T[lang]["resume"], 95, 115, 7)
877         py16.text(T[lang]["map_btn"], 95, 125, 7)
878     elif goal_reached:
879         py16.rectfill(30, 90, 196, 50, 0); py16.rect(30, 90, 196, 50, 7)
880         if current_level == 4:
881             py16.text(T[lang]["game_clear1"], 50, 98, 10)
882             py16.text(T[lang]["game_clear2"], 60, 110, 11)
883         else:
884             py16.text(T[lang]["goal"], 80, 98, 10)
885             py16.text(f"{T[lang]['score']}{score}", 76, 110, 11)
886             py16.text(T[lang]["enter_map"], 75, 125, 7)
887     elif game_state not in ["ENDING", "DYING"]:
888         py16.text(f"{T[lang]['score']}{score}", 4, 4, 10)
889
890 if __name__ == "__main__":
891     py16.run(update, draw, init)
892
```